

# BLOOMBERG LP

## Software Engineer Interview 30-Day Python Preparation Guide

200 Questions · Python Solutions · Bloomberg-Focused

Based on Real 2024–2026 Interview Reports & LeetCode Tag Analysis

Prepared: April 2026

|     |                 |     |      |     |        |     |      |
|-----|-----------------|-----|------|-----|--------|-----|------|
| 200 | Total Questions | 28% | Easy | 61% | Medium | 11% | Hard |
|-----|-----------------|-----|------|-----|--------|-----|------|

# SECTION 1 · BLOOMBERG INTERVIEW OVERVIEW

Bloomberg LP conducts one of the most structured software engineering interview processes in finance-tech. Based on verified candidate reports from 2024–2026, the pipeline is 3–7 weeks long and places roughly 70% of its weight on raw coding ability. Crucially, Bloomberg interviewers do **not** require your code to compile — they care about logical correctness, optimal complexity, and clear communication while you code.

## Interview Pipeline

| Stage                     | Format          | Duration  | Key Focus                                              |
|---------------------------|-----------------|-----------|--------------------------------------------------------|
| 1. Recruiter Screen       | Phone call      | 30 min    | Background, motivation, comp range, 'Why Bloomberg?'   |
| 2. Technical Phone Screen | Remote coding   | 45–60 min | 1–2 LeetCode Medium; must explain every step aloud     |
| 3. Virtual Onsite Rd 1    | Remote coding   | 60 min    | 2 Medium/Hard problems; BFS/DFS, DP, intervals         |
| 4. Virtual Onsite Rd 2    | Code Review     | 60 min    | Spot bugs in 200+ lines of buggy C++/Python code       |
| 5. Virtual Onsite Rd 3    | System Design   | 60 min    | Real-time data feeds, order books, streaming pipelines |
| 6. Behavioral / HR        | Structured STAR | 30–45 min | Collaboration, ownership, 'Why Bloomberg?' (critical!) |

## What Makes Bloomberg Different

- **Code Review Round is unique** — you receive intentionally buggy code (iterator misuse, uninitialised booleans, boundary errors) and must critique it like a senior engineer reviewing a PR from a junior developer.
- **'Why Bloomberg?' is critical** — candidates who solved every problem perfectly have been rejected because they lacked genuine interest in Bloomberg's mission of financial data transparency.
- **No IDE autocomplete** — Bloomberg uses HackerRank-style bare editors. Practice without autocomplete to build muscle memory.
- **Python is fully accepted** — Python is one of Bloomberg's preferred languages alongside C++. All solutions in this guide use Python 3.
- **Communication = 50% of score** — Think out loud. State complexity before and after every optimisation. Narrate edge cases. Silence is expensive.
- **Finance context matters** — Questions are often framed around ticks, order books, trading data, and real-time streams. Understand these patterns.

■ Bloomberg interviewers rank communication equal to technical ability. Multiple 2025 candidates who solved all problems were rejected solely due to poor communication or weak 'Why Bloomberg?' answers.

## SECTION 2 · RECOMMENDED BOOKS & RESOURCES

The books below are ranked by Bloomberg-relevance. All Python-compatible editions are noted. Read them in the order listed — Core tier first, then Supplemental.

### TIER 1 — CORE (Must-Read for Bloomberg Python)

#### Cracking the Coding Interview, 2nd Ed. (2025)

★★★★★  
BLOOMBERG-CRITICAL

*Gayle Laakmann McDowell, Aline Lerner et al.*

The 2025 edition switches all code examples to Python (from Java). Contains ~230 problems shaped by data from 100,000+ FAANG mock interviews. Adds 13 new topics: sliding windows, prefix arrays, rolling hashes. The job-search section (~150 pages) covers offer timing and negotiation — relevant for Bloomberg's structured offer process.

#### Elements of Programming Interviews in Python (EPI Python)

★★★★★  
BLOOMBERG-CRITICAL

*Aziz, Lee, Prakash*

300 problems with full Python solutions. Covers every topic Bloomberg tests: heaps, BSTs, graphs, DP, concurrency. Problems are drawn from real-world applications — financial data, distributed systems — mirroring Bloomberg's style. Recommended by Google/Facebook engineers; ideal for Bloomberg's harder onsite rounds.

#### Grokking the Coding Interview: Patterns (Educative.io)

★★★★★ PATTERN  
FOUNDATION

*Design Gurus*

26 coding patterns (sliding window, two pointers, fast/slow pointers, merge intervals, top-K elements, BFS/DFS, DP). Bloomberg's interview pattern distribution maps almost 1-to-1 to this curriculum. Available in Python. The intervals and top-K patterns appear in virtually every Bloomberg phone screen. Essential pattern-first learning.

### TIER 2 — SUPPLEMENTAL (System Design + Python Depth)

#### Designing Data-Intensive Applications

*Martin Kleppmann* — The definitive guide to distributed systems — directly relevant to Bloomberg's system design rounds on real-time market data, Kafka-style feeds, and consistency trade-offs. Read Chapters 1, 5, 7, 11 (streaming).

#### System Design Interview Vol. 1 & 2

*Alex Xu* — Practical walk-throughs of URL shortener, notification system, stock exchange, rate limiter — all Bloomberg system design archetypes. Chapter on 'Design a Stock Exchange' is must-read.

#### Python Tricks: The Book

*Dan Bader* — Bloomberg interviewers expect idiomatic Python. This book covers list/dict comprehensions, generators, decorators, collections.Counter, heapq, defaultdict — all Python tools you'll use in Bloomberg solutions.

## **Data Structures & Algorithms in Python**

*Goodrich, Tamassia, Goldwasser* — Academic-level Python DS&A; reference. Use for trees, heaps, graph algorithms. Useful for Bloomberg's harder onsite rounds when you need to reason about time complexity rigorously.

## **Silicon Valley Python Engineer Interview Guide (2025)**

*Springer Nature* — Released March 2025. Python-native interview guide covering DS&A; and system design. Directly aligned with 2025-era finance/tech interview patterns.

■ For LeetCode practice, enable 'Bloomberg' company filter (Premium). Use DSAPrep.dev for spaced-repetition tracking of Bloomberg-tagged problems.

# SECTION 3 · BLOOMBERG TOPIC FREQUENCY BREAKDOWN

Analysis of 433 real Bloomberg interview questions (2023–2026) reveals the following topic distribution. Master Tier 1 first — it covers 40–50% of all Bloomberg questions. Tier 2 covers another 20–30%. Tier 3 is targeted preparation for senior roles.

| #  | Topic                   | % of Qs | Tier | Key Bloomberg Patterns                                |
|----|-------------------------|---------|------|-------------------------------------------------------|
| 1  | Arrays & Strings        | 22%     | 1    | Two-pointer, sliding window, prefix sum               |
| 2  | Hash Maps & Sets        | 18%     | 1    | Frequency count, anagram grouping, LRU cache          |
| 3  | Linked Lists            | 9%      | 1    | Reverse, cycle detection, merge sorted lists          |
| 4  | Trees (Binary / BST)    | 12%     | 1    | BFS level-order, DFS, validate BST, kth largest       |
| 5  | Graphs & BFS/DFS        | 10%     | 2    | Number of islands, currency conversion, all paths DAG |
| 6  | Dynamic Programming     | 8%      | 2    | Kadane's, word break, coin change, knapsack           |
| 7  | Heaps & Priority Queues | 6%      | 2    | Top-K elements, median of stream, task scheduler      |
| 8  | Stacks & Queues         | 5%      | 2    | Valid parentheses, daily temperatures, LRU cache      |
| 9  | Sorting & Searching     | 5%      | 2    | Merge intervals, binary search variants               |
| 10 | Design (OOP / System)   | 3%      | 3    | LRU cache, ordered stream, underground system         |
| 11 | Backtracking            | 2%      | 3    | Permutations, subsets, word search                    |

■ Bloomberg emphasises Arrays+Hash Maps (~40% combined). Solve every array/string problem before moving to graphs or DP.

# SECTION 4 · 30-DAY BLOOMBERG PYTHON PREP SCHEDULE

This schedule distributes 200 questions across 30 days: ~7 questions/day on weekdays and 5 questions on weekends, with review and mock sessions built in. All solutions should be written in Python 3. Follow the 3-step method for every problem: (1) clarify, (2) brute force aloud, (3) optimise and code.

■ Daily target: 6–7 problems on weekdays · 4–5 on weekends · Reviews on Day 8, 15, 22, 29

## WEEK 1 · FOUNDATIONS — Arrays, Strings, Hash Maps (Days 1–7 · 50 Qs)

**Goal:** Solve 70%+ of Bloomberg Easy problems in under 10 minutes. Build Python fluency with lists, dicts, sets, collections.Counter, and heapq.

| Day | #   | Problem Name                         | Diff | Topic        | Freq          |
|-----|-----|--------------------------------------|------|--------------|---------------|
| 1   | 1   | Two Sum                              | Easy | Hash Map     | ★★★ Very High |
| 1   | 49  | Group Anagrams                       | Med  | String+Hash  | ★★★ Very High |
| 1   | 217 | Contains Duplicate                   | Easy | Array+Set    | ★★ High       |
| 1   | 242 | Valid Anagram                        | Easy | String+Hash  | ★★ High       |
| 1   | 347 | Top K Frequent Elements              | Med  | Heap/Hash    | ★★★ Very High |
| 1   | 350 | Intersection of Two Arrays II        | Easy | Array        | ★★ High       |
| 2   | 3   | Longest Substring Without Repeating  | Med  | Sliding Win  | ★★★ Very High |
| 2   | 238 | Product of Array Except Self         | Med  | Array        | ★★★ Very High |
| 2   | 167 | Two Sum II — Sorted                  | Med  | Two Pointer  | ★★ High       |
| 2   | 128 | Longest Consecutive Sequence         | Med  | Hash Set     | ★★★ Very High |
| 2   | 560 | Subarray Sum Equals K                | Med  | Prefix Sum   | ★★★ Very High |
| 2   | 659 | Split Array into Consec Subsequences | Med  | Greedy/Hash  | ★★ High       |
| 3   | 15  | 3Sum                                 | Med  | Two Pointer  | ★★★ Very High |
| 3   | 11  | Container With Most Water            | Med  | Two Pointer  | ★★★ Very High |
| 3   | 42  | Trapping Rain Water                  | Hard | Two Pointer  | ★★ High       |
| 3   | 443 | String Compression                   | Med  | String       | ★★ High       |
| 3   | 412 | FizzBuzz                             | Easy | String       | ★ Med         |
| 3   | 8   | String to Integer (atoi)             | Med  | String       | ★★★ Very High |
| 4   | 392 | Is Subsequence                       | Easy | Two Pointer  | ★★ High       |
| 4   | 438 | Find All Anagrams in a String        | Med  | Sliding Win  | ★★★ Very High |
| 4   | 567 | Permutation in String                | Med  | Sliding Win  | ★★★ Very High |
| 4   | 394 | Decode String                        | Med  | Stack/String | ★★★ Very High |
| 4   | 387 | First Unique Character in String     | Easy | Hash Map     | ★★ High       |
| 4   | 151 | Reverse Words in a String            | Med  | String       | ★★ High       |
| 5   | 239 | Sliding Window Maximum               | Hard | Deque        | ★★★ Very High |

|   |               |                                      |             |              |                   |
|---|---------------|--------------------------------------|-------------|--------------|-------------------|
| 5 | <b>76</b>     | Minimum Window Substring             | <b>Hard</b> | Sliding Win  | ★★ High           |
| 5 | <b>209</b>    | Minimum Size Subarray Sum            | <b>Med</b>  | Sliding Win  | ★★ High           |
| 5 | <b>480</b>    | Sliding Window Median                | <b>Hard</b> | Heap         | ★★ High           |
| 5 | <b>159</b>    | Longest Substr w/ At Most 2 Distinct | <b>Med</b>  | Sliding Win  | ★★ High           |
| 6 | <b>169</b>    | Majority Element                     | <b>Easy</b> | Array        | ★★ High           |
| 6 | <b>189</b>    | Rotate Array                         | <b>Med</b>  | Array        | ★★ High           |
| 6 | <b>41</b>     | First Missing Positive               | <b>Hard</b> | Array        | ★★ High           |
| 6 | <b>73</b>     | Set Matrix Zeroes                    | <b>Med</b>  | Array/Matrix | ★★ High           |
| 6 | <b>54</b>     | Spiral Matrix                        | <b>Med</b>  | Array/Matrix | ★★★ Very High     |
| 7 | <b>REVIEW</b> | Re-solve Days 1–3 from memory        | —           | All          | Spaced Repetition |
| 7 | <b>MOCK</b>   | Timed mock: 2 problems, 45 min       | —           | All          | Mock Interview    |

**Week 1 Python Tools:** Use `collections.Counter`, `collections.defaultdict`, `set()`, and `dict.get(k, 0)` for hash map problems. For sliding window, maintain a left pointer and a running count dict.

## WEEK 2 · LINKED LISTS, STACKS, TREES (Days 8–14 · 50 Qs)

**Goal:** Solve tree BFS/DFS problems under 20 minutes. Master Python's `collections.deque` for BFS and recursive DFS templates.

| Day | #      | Problem Name                          | Diff | Topic       | Freq              |
|-----|--------|---------------------------------------|------|-------------|-------------------|
| 8   | REVIEW | Re-solve Week 1 favourites (pick 5)   | —    | All         | Spaced Repetition |
| 8   | 206    | Reverse Linked List                   | Easy | Linked List | ★★★ Very High     |
| 8   | 21     | Merge Two Sorted Lists                | Easy | Linked List | ★★★ Very High     |
| 8   | 141    | Linked List Cycle                     | Easy | Linked List | ★★★ Very High     |
| 8   | 160    | Intersection of Two Linked Lists      | Easy | Linked List | ★★★ Very High     |
| 9   | 2      | Add Two Numbers                       | Med  | Linked List | ★★★ Very High     |
| 9   | 19     | Remove Nth Node From End              | Med  | Linked List | ★★★ Very High     |
| 9   | 61     | Rotate List                           | Med  | Linked List | ★★ High           |
| 9   | 138    | Copy List with Random Pointer         | Med  | Linked List | ★★ High           |
| 9   | 430    | Flatten Multilevel Doubly Linked List | Med  | Linked List | ★★★ Very High     |
| 10  | 20     | Valid Parentheses                     | Easy | Stack       | ★★★ Very High     |
| 10  | 155    | Min Stack                             | Med  | Stack       | ★★★ Very High     |
| 10  | 739    | Daily Temperatures                    | Med  | Stack       | ★★★ Very High     |
| 10  | 84     | Largest Rectangle in Histogram        | Hard | Stack       | ★★ High           |
| 10  | 232    | Implement Queue Using Stacks          | Easy | Stack/Queue | ★★★ Very High     |
| 11  | 102    | Binary Tree Level Order Traversal     | Med  | Tree BFS    | ★★★ Very High     |
| 11  | 199    | Binary Tree Right Side View           | Med  | Tree BFS    | ★★★ Very High     |
| 11  | 104    | Max Depth of Binary Tree              | Easy | Tree DFS    | ★★ High           |
| 11  | 226    | Invert Binary Tree                    | Easy | Tree DFS    | ★★ High           |
| 11  | 112    | Path Sum                              | Easy | Tree DFS    | ★★ High           |
| 12  | 98     | Validate Binary Search Tree           | Med  | BST         | ★★★ Very High     |
| 12  | 230    | Kth Smallest Element in BST           | Med  | BST         | ★★★ Very High     |
| 12  | 235    | LCA of Binary Search Tree             | Med  | BST         | ★★★ Very High     |
| 12  | 105    | Construct BT from Preorder/Inorder    | Med  | Tree        | ★★ High           |
| 12  | 124    | Binary Tree Max Path Sum              | Hard | Tree DFS    | ★★ High           |
| 13  | 297    | Serialize/Deserialize Binary Tree     | Hard | Tree        | ★★ High           |
| 13  | 543    | Diameter of Binary Tree               | Easy | Tree DFS    | ★★ High           |
| 13  | 572    | Subtree of Another Tree               | Easy | Tree        | ★★ High           |
| 13  | 114    | Flatten BT to Linked List             | Med  | Tree        | ★★ High           |
| 14  | REVIEW | Re-solve Days 8–11 from memory        | —    | All         | Spaced Repetition |



|    |                        |                                |   |     |                |
|----|------------------------|--------------------------------|---|-----|----------------|
| 14 | <b>MOC</b><br><b>K</b> | Timed mock: 2 problems, 50 min | — | All | Mock Interview |
|----|------------------------|--------------------------------|---|-----|----------------|

**Week 2 Python Tools:** BFS with `from collections import deque; q = deque([root])`. DFS recursion with base case if not node: return. For BST validation, pass `min_val`, `max_val` bounds down the recursion.

## WEEK 3 · GRAPHS, HEAPS, DYNAMIC PROGRAMMING (Days 15–21 · 60 Qs)

**Goal:** Recognise Bloomberg-specific graph patterns (currency conversion, org chart, trading graph). Solve DP problems using tabulation. Use heapq for all heap operations in Python.

| Day | #             | Problem Name                        | Diff | Topic         | Freq              |
|-----|---------------|-------------------------------------|------|---------------|-------------------|
| 15  | <b>REVIEW</b> | Re-solve Week 2 favourites (pick 5) | —    | All           | Spaced Repetition |
| 15  | <b>200</b>    | Number of Islands                   | Med  | Graph BFS/DFS | ★★★ Very High     |
| 15  | <b>133</b>    | Clone Graph                         | Med  | Graph DFS     | ★★★ Very High     |
| 15  | <b>695</b>    | Max Area of Island                  | Med  | Graph DFS     | ★★ High           |
| 16  | <b>207</b>    | Course Schedule                     | Med  | Graph/Topo    | ★★★ Very High     |
| 16  | <b>210</b>    | Course Schedule II                  | Med  | Graph/Topo    | ★★★ Very High     |
| 16  | <b>310</b>    | Minimum Height Trees                | Med  | Graph         | ★★ High           |
| 16  | <b>787</b>    | Cheapest Flights Within K Stops     | Med  | Graph/DP      | ★★ High           |
| 16  | <b>BLOOM</b>  | Currency Conversion Rates (Custom)  | Med  | Graph BFS     | ★★★ Bloomberg!    |
| 17  | <b>417</b>    | Pacific Atlantic Water Flow         | Med  | Graph DFS     | ★★ High           |
| 17  | <b>127</b>    | Word Ladder                         | Hard | Graph BFS     | ★★★ Very High     |
| 17  | <b>743</b>    | Network Delay Time (Dijkstra)       | Med  | Graph/Heap    | ★★★ Very High     |
| 17  | <b>332</b>    | Reconstruct Itinerary               | Hard | Graph/Euler   | ★★ High           |
| 17  | <b>269</b>    | Alien Dictionary                    | Hard | Topo Sort     | ★★ High           |
| 18  | <b>215</b>    | Kth Largest Element in Array        | Med  | Heap          | ★★★ Very High     |
| 18  | <b>347</b>    | Top K Frequent Words                | Med  | Heap          | ★★★ Very High     |
| 18  | <b>295</b>    | Find Median from Data Stream        | Hard | Heap          | ★★★ Very High     |
| 18  | <b>373</b>    | Find K Pairs with Smallest Sums     | Med  | Heap          | ★★ High           |
| 18  | <b>692</b>    | Top K Frequent Words                | Med  | Heap/Sort     | ★★ High           |
| 19  | <b>23</b>     | Merge K Sorted Lists                | Hard | Heap          | ★★★ Very High     |
| 19  | <b>621</b>    | Task Scheduler                      | Med  | Heap/Greedy   | ★★★ Very High     |
| 19  | <b>703</b>    | Kth Largest in a Stream             | Easy | Heap          | ★★★ Very High     |
| 19  | <b>BLOOM</b>  | Top K Traded Stocks (Custom)        | Med  | Heap          | ★★★ Bloomberg!    |
| 20  | <b>53</b>     | Maximum Subarray (Kadane's)         | Med  | DP            | ★★★ Very High     |
| 20  | <b>70</b>     | Climbing Stairs                     | Easy | DP            | ★★ High           |
| 20  | <b>198</b>    | House Robber                        | Med  | DP            | ★★ High           |
| 20  | <b>322</b>    | Coin Change                         | Med  | DP            | ★★★ Very High     |
| 20  | <b>139</b>    | Word Break                          | Med  | DP            | ★★★ Very High     |
| 21  | <b>62</b>     | Unique Paths                        | Med  | DP/2D         | ★★ High           |
| 21  | <b>72</b>     | Edit Distance                       | Hard | DP            | ★★ High           |

|    |               |                                 |     |     |                   |
|----|---------------|---------------------------------|-----|-----|-------------------|
| 21 | <b>300</b>    | Longest Increasing Subsequence  | Med | DP  | ★★★ Very High     |
| 21 | <b>REVIEW</b> | Re-solve Days 15–18 from memory | —   | All | Spaced Repetition |
| 21 | <b>MOC K</b>  | Timed mock: 2 problems, 55 min  | —   | All | Mock Interview    |

**Week 3 Python Tools:** Graph with adjacency list: `graph = defaultdict(list)`. BFS: `from collections import deque`. Heap: `import heapq; heapq.heappush(heap, val); heapq.heappop(heap)`. Min-heap by default; for max-heap push `-val`. DP: always define base cases first; prefer bottom-up tabulation in interviews.

## WEEK 4 · BLOOMBERG-SPECIFIC + MOCK + SYSTEM DESIGN (Days 22–30 · 40 Qs)

**Goal:** Focus on Bloomberg's unique rounds — Code Review, Design, and Finance-framed problems. Run 4 full mock interview sessions. Finalise 'Why Bloomberg?' narrative.

| Day | #      | Problem Name                             | Diff | Topic          | Freq              |
|-----|--------|------------------------------------------|------|----------------|-------------------|
| 22  | REVIEW | Re-solve all ★★★ problems from Weeks 1–3 | —    | All            | Spaced Repetition |
| 22  | 56     | Merge Intervals                          | Med  | Array/Sort     | ★★★ Very High     |
| 22  | 57     | Insert Interval                          | Med  | Array          | ★★★ Very High     |
| 22  | 253    | Meeting Rooms II                         | Med  | Heap/Intervals | ★★★ Very High     |
| 23  | 380    | Insert Delete GetRandom O(1)             | Med  | Hash Map       | ★★★ Very High     |
| 23  | 381    | Insert Delete GetRandom (Duplicates)     | Hard | Hash Map       | ★★★ Very High     |
| 23  | 146    | LRU Cache                                | Med  | Design         | ★★★ Very High     |
| 23  | 460    | LFU Cache                                | Hard | Design         | ★★ High           |
| 24  | 341    | Flatten Nested List Iterator             | Med  | Design/Stack   | ★★ High           |
| 24  | 284    | Peeking Iterator                         | Med  | Design         | ★★ High           |
| 24  | 1396   | Design Underground System                | Med  | Design/Hash    | ★★★ Very High     |
| 24  | 1472   | Design Browser History                   | Med  | Design         | ★★ High           |
| 25  | BLOOM  | Ordered Stream of Ticks (Custom)         | Med  | Design         | ★★★ Bloomberg!    |
| 25  | BLOOM  | First Unique in Data Stream (Custom)     | Med  | Design/Queue   | ★★★ Bloomberg!    |
| 25  | BLOOM  | Org Chart Topological Sort (Custom)      | Med  | Graph/Topo     | ★★★ Bloomberg!    |
| 25  | BLOOM  | Invalid Transactions (LC 1169)           | Med  | String/Sort    | ★★★ Bloomberg!    |
| 26  | MOC K  | Full Mock Round 1: 2 Medium, 50 min      | —    | All            | Mock Interview    |
| 27  | MOC K  | Full Mock Round 2: Code Review Session   | —    | Code Review    | Mock Interview    |
| 28  | MOC K  | Full Mock Round 3: System Design 60 min  | —    | Sys Design     | Mock Interview    |
| 29  | REVIEW | Spaced repetition: all flagged problems  | —    | All            | Final Review      |
| 30  | MOC K  | Full Mock Round 4: 3-round Bloomberg sim | —    | All            | Final Simulation  |

■ For the Code Review mock (Day 27): find a buggy Python script online, set a 20-minute timer, and narrate every bug you find out loud as if presenting to a Bloomberg team.

# SECTION 5 · TOP 20 HIGHEST-FREQUENCY BLOOMBERG QUESTIONS

The following 20 problems appear across the highest number of Bloomberg interview reports (2023–2026). Every Bloomberg candidate should be able to solve these fluently in Python.

| #  | LC # | Problem                           | Diff | Key Python Insight                          |
|----|------|-----------------------------------|------|---------------------------------------------|
| 1  | 380  | Insert Delete GetRandom O(1)      | Med  | dict + list; swap-remove trick              |
| 2  | 56   | Merge Intervals                   | Med  | Sort by start; compare end vs next start    |
| 3  | 347  | Top K Frequent Elements           | Med  | heapq.nlargest(k, count, key=count.get)     |
| 4  | 146  | LRU Cache                         | Med  | OrderedDict or doubly-linked list + dict    |
| 5  | 3    | Longest Substr Without Repeating  | Med  | Sliding window with set; move left on dup   |
| 6  | 1    | Two Sum                           | Easy | dict{val: idx}; check complement first      |
| 7  | 394  | Decode String                     | Med  | Stack of (count, built_str)                 |
| 8  | 49   | Group Anagrams                    | Med  | defaultdict; key = tuple(sorted(word))      |
| 9  | 102  | Binary Tree Level Order Traversal | Med  | deque BFS; snapshot len(queue) per level    |
| 10 | 295  | Find Median from Data Stream      | Hard | Max-heap (lo) + min-heap (hi); balance      |
| 11 | 207  | Course Schedule                   | Med  | DFS cycle detection; visited states: 0/1/2  |
| 12 | 200  | Number of Islands                 | Med  | DFS; mark visited in-place with '0'         |
| 13 | 739  | Daily Temperatures                | Med  | Mono-decreasing stack of indices            |
| 14 | 53   | Maximum Subarray                  | Med  | Kadane: max_end = max(n, max_end + n)       |
| 15 | 139  | Word Break                        | Med  | dp[i] = any(dp[j] and s[j:i] in wordSet)    |
| 16 | 8    | String to Integer (atoi)          | Med  | Strip, sign, digit loop; clamp to INT range |
| 17 | 23   | Merge K Sorted Lists              | Hard | heapq with (val, idx, node)                 |
| 18 | 199  | Binary Tree Right Side View       | Med  | BFS; append last node of each level         |
| 19 | 98   | Validate BST                      | Med  | Recursive with (min, max) bounds            |
| 20 | 127  | Word Ladder                       | Hard | BFS with wildcard neighbours; visited set   |

# SECTION 6 · BLOOMBERG-SPECIFIC CUSTOM QUESTIONS

Bloomberg regularly asks custom finance-flavoured problems not on LeetCode. Below are the most frequently reported custom problems with Python solution sketches.

## 1. Currency Conversion Rates (Graph BFS)

Build a graph where edges are conversion rates. Given FROM and TO currencies, find the conversion factor using BFS/DFS. E.g., USD→JPY→GBP.

```
# Input: rates=[('USD','JPY',110),('JPY','GBP',0.007)], query=('USD','GBP') # Graph: {USD: [(JPY, 110)], JPY: [(USD, 1/110), (GBP, 0.007)]} # BFS: multiply rates along the path from
collections import defaultdict, deque
def convert(rates, src, dst):
    graph = defaultdict(list)
    for a, b, r in rates:
        graph[a].append((b, r))
        graph[b].append((a, 1/r))
    q = deque([(src, 1.0)])
    visited = {src}
    while q:
        node, rate = q.popleft()
        if node == dst:
            return rate
        for nei, r in graph[node]:
            if nei not in visited:
                visited.add(nei)
                q.append((nei, rate * r))
    return -1
```

## 2. Top K Traded Stocks (Heap + Stream)

You receive a stream of (stock\_name, volume) ticks throughout the day. Return top-K stocks by total volume at any point. Bloomberg loves heap + hashmap combos.

```
import heapq
from collections import defaultdict
class StockTracker:
    def __init__(self, k):
        self.k = k
        self.vol = defaultdict(int)
    def addTick(self, name, vol):
        self.vol[name] += vol
    def topK(self):
        return heapq.nlargest(self.k, self.vol, key=self.vol.get)
```

## 3. First Unique Number in Data Stream

Design a data structure supporting add(num) and firstUnique() in O(1) each. Use an OrderedDict to maintain insertion order with O(1) delete.

```
from collections import OrderedDict
class UniqueStream:
    def __init__(self):
        self.unique = OrderedDict()
        self.dup = set()
    def add(self, n):
        if n in self.dup:
            return
        if n in self.unique:
            del self.unique[n]
        self.dup.add(n)
        self.unique[n] = True
    def firstUnique(self):
        return next(iter(self.unique)) if self.unique else -1
```

## 4. Org Chart Topological Sort

Given a list of (name, title) pairs and a dict of {title: reports\_to\_title}, sort employees from lowest to highest in the hierarchy.

```
# Input: employees=[('John','Manager'),('Sally','CTO'),...] # hierarchy={Engineer: Manager, Manager: CTO, CTO: CEO, CFO: CEO} # Build level map via BFS from leaves, sort by level
```

## 5. Ordered Stream of Ticks (LC 1656 variant)

n ticks arrive out of order. idKey is 1..n. Return streaks of consecutive ticks as they become available. Use a buffer dict and a 'ptr' pointer.

```
class OrderedStream: def __init__(self, n): self.buf = {}; self.ptr = 1 def insert(self, id, val): self.buf[id] = val; res = [] while self.ptr in self.buf: res.append(self.buf[self.ptr]); self.ptr += 1 return res
```

# SECTION 7 · BLOOMBERG SYSTEM DESIGN GUIDE

Bloomberg's system design round focuses on real-time financial data infrastructure. Unlike FAANG which tests generic internet-scale systems, Bloomberg asks about low-latency trade data pipelines, order books, and market feeds.

| Topic                        | Key Components to Discuss                       | Bloomberg Angle                              |
|------------------------------|-------------------------------------------------|----------------------------------------------|
| Real-Time Market Data Feed   | Pub/Sub (Kafka), sequence numbers, WebSocket    | Millisecond latency SLAs; warm failover      |
| Order Book                   | Max-heap bids, min-heap asks, matching engine   | ACID persistence; audit trail; concurrency   |
| Stock Price Alert System     | Priority queue, sliding window, threshold check | Streaming aggregation; fan-out notifications |
| Time-Series Database         | LSM tree, column store, compression             | Tick data: 1M+ writes/sec; rollup queries    |
| URL Shortener / Audit Tool   | Base-62 hash, Redis cache, click stream Kafka   | Bloomberg terminal deep links; click logging |
| Distributed Stream Processor | Kafka consumers, checkpointing, idempotency     | Position reconciliation; EOD batch           |

■ Bloomberg's 'Why Bloomberg?' system design angle: always mention how your design serves financial professionals — trader latency requirements, audit compliance, global data normalisation.



## SECTION 8 · THE CODE REVIEW ROUND (Bloomberg-Unique)

Bloomberg is the only major tech company with a dedicated Code Review interview round. You receive 200+ lines of intentionally buggy code and must critique it like a PR review. This round is as important as the coding rounds — failing it equals rejection.

### Common Bug Categories Bloomberg Plants

- **Iterator Misuse:** Comparing iterator to wrong endpoint, e.g. if it `== map.end()` when it should be `container.end()`
- **Uninitialised Variables:** bool flag; used before assignment. Python: variable referenced before assignment in branch
- **Off-by-One Errors:** for i in `range(n)` vs `range(n+1)`; array index n vs n-1 at boundary
- **Logic Errors:** Wrong comparison operator (`<=` vs `<`); incorrect base case in recursion
- **Memory/Resource Leaks:** (C++) Not releasing allocated resources. Python: unclosed file handles, unclosed DB connections
- **Concurrency Issues:** Shared state without locks in threaded code; non-atomic read-modify-write
- **Edge Cases Missed:** Empty input, single element, negative numbers, integer overflow not handled
- **Wrong Algorithm:**  $O(n^2)$  where  $O(n \log n)$  was possible; quadratic search in inner loop

### How to Approach the Code Review Round

Treat it exactly like reviewing a real PR. Narrate out loud using this structure:

- 1. First pass (2 min): Understand what the code is supposed to do — ask if unclear.
- 2. Second pass (5 min): Scan for obvious bugs — uninitialised vars, wrong loop bounds, missing null checks.
- 3. Third pass (10 min): Trace through with a small example input. Write down actual vs expected output.
- 4. Fourth pass (5 min): Comment on style, complexity, and missing test coverage.
- 5. Always state: 'I would add a unit test for X edge case' — Bloomberg values testability.

■■ Do NOT stay silent during the code review. Bloomberg interviewers expect a continuous verbal stream of observations. Silence for >2 minutes is a red flag.

## SECTION 9 · BEHAVIORAL PREP & 'WHY BLOOMBERG?'

Bloomberg's behavioral round is not optional filler — it determines your offer. Candidates who solve every technical problem perfectly have been rejected for weak behavioral answers or a generic 'Why Bloomberg?' response. Prepare 5 STAR stories and a specific, genuine Bloomberg mission statement.

### STAR Stories to Prepare (5 Required)

| Story Theme                           | Bloomberg Virtue Being Tested        |
|---------------------------------------|--------------------------------------|
| Led a project under tight deadline    | Ownership + execution under pressure |
| Resolved conflict with teammate       | Collaboration + communication        |
| Identified and fixed a production bug | Engineering discipline + initiative  |
| Improved system performance           | Technical depth + impact mindset     |
| Adapted when requirements changed     | Flexibility + stakeholder management |

### 'Why Bloomberg?' — The Non-Negotiable

Your answer must be specific to Bloomberg's mission. Generic 'big company + good pay' answers are disqualifying.

- **Strong answer elements:**

- Reference Bloomberg's role in financial data transparency — 'Bloomberg Terminal democratized access to market data that was previously only available to insiders.'
- Mention the engineering challenge — 'Real-time, low-latency systems processing billions of ticks per day are a different class of engineering problem from typical web apps.'
- Connect to your background — 'My experience with streaming data pipelines at [X] maps directly to how Bloomberg's market data infrastructure works.'
- Show awareness of Bloomberg's culture — 'Bloomberg's commitment to philanthropic initiatives and data journalism through Bloomberg News aligns with my values.'

■ Never say: 'Bloomberg is prestigious' or 'good pay and benefits'. These answers have historically resulted in rejection regardless of technical performance.

# SECTION 10 · PYTHON CHEAT SHEET FOR BLOOMBERG

All Python patterns you will use in Bloomberg interviews. Memorise these before your first round.

## Hash Map / Counter

```
from collections import Counter, defaultdict
count = Counter(s) # {'a':3, 'b':1, ...}
freq = defaultdict(int) # freq[key] += 1
```

## Sliding Window

```
left = 0; window = {}
for right, c in enumerate(s):
    window[c] = window.get(c, 0) + 1
    while condition_violated:
        window[s[left]] -= 1; left += 1
```

## BFS (Graph / Tree)

```
from collections import deque
q = deque([start]); visited = {start}
while q:
    node = q.popleft()
    for nei in graph[node]:
        if nei not in visited:
            visited.add(nei); q.append(nei)
```

## Min-Heap / Max-Heap

```
import heapq
heap = []
heapq.heappush(heap, val) # min-heap
smallest = heapq.heappop(heap) # Max-heap: push -val, pop and negate
heapq.nlargest(k, iterable, key=fn)
```

## Two Pointers

```
left, right = 0, len(arr)-1
while left < right:
    if condition: left += 1
    else: right -= 1
```

## DFS Recursive Template

```
def dfs(node, visited):
    if not node or node in visited: return
    visited.add(node)
    for nei in graph[node]:
        dfs(nei, visited)
```

## Binary Search

```
import bisect
lo, hi = 0, len(arr)-1
while lo <= hi:
    mid = (lo+hi)//2
    if arr[mid] == target: return mid
    elif arr[mid] < target: lo = mid+1
    else: hi = mid-1
```

## LRU Cache (OrderedDict)

```
from collections import OrderedDict
class LRU:
    def __init__(self, cap):
        self.cap = cap; self.cache = OrderedDict()
    def get(self, k):
        if k not in self.cache: return -1
        self.cache.move_to_end(k); return self.cache[k]
    def put(self, k, v):
        if k in self.cache: self.cache.move_to_end(k)
        self.cache[k] = v
        if len(self.cache) > self.cap:
            self.cache.popitem(last=False)
```

## Merge Intervals

```
intervals.sort(key=lambda x: x[0])
merged = [intervals[0]]
for s, e in intervals[1:]:
    if s <= merged[-1][1]:
        merged[-1][1] = max(merged[-1][1], e)
    else: merged.append([s, e])
```

## Top-K with Heap

```
import heapq
from collections import Counter
count = Counter(nums)
return heapq.nlargest(k, count.keys(), key=count.get)
```

## SECTION 11 · DAILY ROUTINE & MILESTONES

| Time Slot  | Activity                                                       | Duration |
|------------|----------------------------------------------------------------|----------|
| Morning    | Warm-up: 1 Easy Bloomberg problem without looking up anything  | 20 min   |
| Morning    | Core session: 2–3 Medium problems; write Python; narrate aloud | 90 min   |
| Afternoon  | Review yesterday's solutions; optimise; add edge case tests    | 30 min   |
| Evening    | Read EPI Python or CtCI 2025 — 1 chapter                       | 45 min   |
| Evening    | Spaced repetition: re-solve 2 flagged problems from past days  | 30 min   |
| Before Bed | Write down 1 pattern insight or Python trick learned today     | 5 min    |

### 30-Day Milestone Checklist

|        |                                                                                                            |
|--------|------------------------------------------------------------------------------------------------------------|
| Day 7  | ■ Completed 50 questions. Comfortable with arrays, strings, and hash maps. 'Why Bloomberg?' draft written. |
| Day 14 | ■ Completed 100 questions. Trees and linked lists solved fluently. Code Review round practised once.       |
| Day 21 | ■ Completed 160 questions. Graphs, heaps, and DP patterns internalised. 2 mock interviews completed.       |
| Day 27 | ■ Completed 190 questions. System design narratives ready. 4 mock interview sessions done.                 |
| Day 30 | ■ 200 questions complete. Full Bloomberg simulation (3 rounds back-to-back). Interview-ready.              |

---

**Final Reminder:** Bloomberg does not require perfect code. They require logical correctness, optimal complexity, clear communication, and genuine enthusiasm for their mission. Code in Python confidently, explain every decision, and show that you understand why Bloomberg's financial data infrastructure problems are uniquely challenging and important.